I have a Galaga project. For Task 1, fix the Bézier enemy movement bug. In bezier(), the p2 terms should use t ** 2. Do not change other game logic.

Yep bro. **Task 1 only**: fix the Bézier calculation so the `p2` terms use `t ** 2`. Nothing else in the game logic changes.

Python Run

```python
def bezier(p0, p1, p2, t):
    return (
        (1 - t) ** 2 * p0
        + 2 * (1 - t) * t * p1
        + t ** 2 * p2
    )
```

The important fix is:

Python Run

```python
+ t ** 2 * p2
```

instead of incorrectly using `t` for the `p2` term.

Now for Task 2, make enemy colors change based on the current wave. Keep the existing enemy_tint(kind) function signature. Use a minimal CURRENT_WAVE approach, update it whenever a new wave starts, and keep the rest of the game logic unchanged.

Yep. For **Task 2**, make only the minimal changes:

Python Run

```python
CURRENT_WAVE = 1
```

Keep the existing signature:

Python Run

```python
def enemy_tint(kind):
    base = ENEMY_COLORS[kind]
    shift = (CURRENT_WAVE - 1) * 20

    return tuple(min(255, c + shift) for c in base)
```

Then, **whenever a new wave starts**, update:

Python Run

```python
CURRENT_WAVE = wave
```

So the flow is:

Python Run

```python
CURRENT_WAVE = 1

def enemy_tint(kind):
    base = ENEMY_COLORS[kind]
    shift = (CURRENT_WAVE - 1) * 20
    return tuple(min(255, c + shift) for c in base)
```

And at the existing **wave-start point**:

Python Run

```python
wave += 1
CURRENT_WAVE = wave
```

This keeps `enemy_tint(kind)` unchanged in signature and leaves the rest of the game logic alone.



Now for Task 3, add a wave-start banner. Whenever a new wave begins, display WAVE <number> in the center of the screen for about 1.5 seconds. Keep existing game logic unchanged apart from the required banner state, timer, update and drawing code.

Yep bro. For **Task 3**, add only the banner state, timer/update logic, and drawing. Keep the existing game logic untouched.

1. Add banner state

Near your other global game state:

Python Run

```python
WAVE_BANNER = ""
WAVE_BANNER_TIMER = 0
WAVE_BANNER_DURATION = 1.5
```

2. When a new wave begins

At the existing point where the new wave starts:

Python Run

```python
CURRENT_WAVE = wave
WAVE_BANNER = f"WAVE {wave}"
WAVE_BANNER_TIMER = WAVE_BANNER_DURATION
```

3. Update the banner timer

Inside the existing game update function, using the existing `dt`:

Python Run

```python
if WAVE_BANNER_TIMER > 0:
    WAVE_BANNER_TIMER -= dt
    if WAVE_BANNER_TIMER <= 0:
        WAVE_BANNER_TIMER = 0
        WAVE_BANNER = ""
```

4. Draw the banner

In the existing drawing/rendering section:

Python Run

```python
if WAVE_BANNER_TIMER > 0:
    banner_surface = font.render(WAVE_BANNER, True, WHITE)
    banner_rect = banner_surface.get_rect(center=(WIDTH // 2, HEIGHT // 2))
    screen.blit(banner_surface, banner_rect)
```

If your project uses a different font variable, screen size variable, or white-color constant, use the **existing ones** rather than creating new game logic.

So the only additions are:

State → wave-start trigger → timer update → centered drawing.

No changes to enemy movement, spawning, shooting, scoring, or other game logic.

Now for Task 4, implement shield_charges(wave) using wave // 3. Do not change other game logic.

For **Task 4**, replace the existing `shield_charges(wave)` implementation with just:

Python Run

```python
def shield_charges(wave):
    return wave // 3
```

Nothing else needs to change. This gives:

- Wave 1–2 → `0` shields
- Wave 3–5 → `1` shield
- Wave 6–8 → `2` shields
- Wave 9–11 → `3` shields
- etc.

ChatGPT can make mistakes. Check important info.